



Week 3 Lab Sheet — Encapsulation in C#



Objectives

- Understand the principle of **encapsulation** in OOP.
- Learn how to **hide data** using access modifiers (`private` , `public`).
- Practice using **properties** and **methods** to control access.
- Apply encapsulation to a practical example (Bank Account).



Theory

- **Encapsulation** = wrapping data and methods together while restricting direct access.
- Use **private fields** to store data safely.
- Provide **public methods/properties** to interact with data.
- Benefits:
 - Protects data integrity.
 - Prevents unauthorized modification.
 - Makes code more maintainable.



Step-by-Step Guidelines

1. Create a new **Console Application** project.
2. Define a `BankAccount` class with a **private balance field**.
3. Add **public methods** for deposit, withdraw, and balance check.
4. Validate inputs (e.g., prevent negative deposits or overdrafts).
5. Test with multiple transactions.



Full Example Code

BankAccount.cs

```
using System;
```

```
public class BankAccount
{
    // Private field
    private decimal balance;

    // Constructor
    public BankAccount(decimal initialBalance)
    {
        if (initialBalance >= 0)
            balance = initialBalance;
        else
            balance = 0;
    }

    // Public method to deposit
    public void Deposit(decimal amount)
    {
        if (amount > 0)
        {
            balance += amount;
            Console.WriteLine($"Deposited: {amount}, New Balance: {balance}");
        }
        else
        {
            Console.WriteLine("Deposit amount must be positive.");
        }
    }

    // Public method to withdraw
    public void Withdraw(decimal amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            Console.WriteLine($"Withdrawn: {amount}, New Balance: {balance}");
        }
        else
        {
            Console.WriteLine("Invalid withdrawal amount.");
        }
    }

    // Public method to check balance
    public decimal GetBalance()
    {
        return balance;
    }
}
```

Program.cs

```
using System;

class Program
{
    static void Main(string[] args)
    {
        // Create account with initial balance
        BankAccount account = new BankAccount(1000);

        // Perform transactions
        account.Deposit(500);
        account.Withdraw(200);
        account.Withdraw(2000); // Invalid
        Console.WriteLine($"Final Balance: {account.GetBalance()}");

        Console.ReadLine();
    }
}
```



Expected Output

```
Deposited: 500, New Balance: 1500
Withdrawn: 200, New Balance: 1300
Invalid withdrawal amount.
Final Balance: 1300
```



Lab Exercises

1. Add a method to **transfer money** between two accounts.
2. Prevent withdrawals if balance < 100 (minimum balance rule).
3. Add a **transaction history list** and display all deposits/withdrawals.
4. Extend with **properties** (`Balance` as read-only property).



Evaluation Criteria

- Correct use of **private fields** (5 marks).
- Proper **public methods** for controlled access (10 marks).
- Input validation (10 marks).

Transfer money method (10 marks)

- Transaction handling (10 marks).
- Extra exercises (15 marks).

Total: 50 marks